Glioblastoma Single-Cell RNA-seq Analysis

Computational Pipeline Documentation

Trajectory Inference and Cellular Differentiation Analysis

Pipeline Overview

This document describes a computational pipeline for analyzing single-cell RNA sequencing data from glioblastoma tumors integrated with normal fetal brain tissue. The pipeline performs batch correction, trajectory inference, pathway enrichment analysis, and neural differential equation modeling to understand tumor cell hierarchies and stem cell differentiation.

Dataset

Tumor data	Human glioblastoma (GSE84465, Darmanis 2017, Smart-seq2)
Normal data	Normal human fetal brain tissue
Integrated cells	~103,426 cells (3,567 tumor + 99,859 normal)
Genes	16,388 (highly variable genes after intersection)
Key markers	SOX2, NES, VIM, FABP7, ASCL1, OLIG2

Pipeline Structure

0a. download.py	Download GBM tumor dataset from NCBI GEO (GSE84465)
0b. cache_normal_brain.py	Subsample 100K cells from normal fetal brain HDF5
1. scVI Training	Batch correction and joint embedding of tumor + normal
2. DPT Analysis	Pseudotime, PAGA, and trajectory inference
3. Enrichment Analysis	Pathway and transcription factor activity
4. VAE Step 1	Custom VAE baseline without balanced sampling
5. VAE Step 2	Stem marker cosine similarity computation
6. VAE Step 3 (Ultra)	VAE + contrastive learning on latent space
7. VAE Step 3 (Contrastive)	Pure contrastive learning without VAE decoder
8. scDiffEq	Neural SDE-based trajectory simulation
9. CONCORD	Contrastive learning-based batch integration and embedding

Execution Order

Before running any notebooks, the two data acquisition scripts (download.py and cache_normal_brain.py) must be executed to prepare the raw datasets. After that, the notebooks should be executed in the order listed above, as each depends on outputs from prior steps. The primary shared file is models/tumor_normal_integrated.h5ad, produced by the scVI notebook and consumed by all subsequent notebooks.

Environment Setup

A conda environment file (env.yml) is provided with the project. This captures all required packages and versions needed to run the full pipeline.

Creating the Environment

Run the following command once to create the conda environment from the provided file:

```
conda env create -f env.yml
```

Then activate it before running any scripts or notebooks:

```
conda activate bio
```

Exporting / Updating the Environment

If you install new packages and want to update the env.yml, run the following with the environment active:

```
conda env export --no-builds > env.yml
```

The --no-builds flag omits platform-specific build strings, making the file portable across operating systems.

Common Dependencies

```
pip install scanpy scvi-tools torch numpy pandas matplotlib seaborn
pip install decoupler concord scdiffeq scikit-learn tqdm plotly
```

0. Data Acquisition and Preprocessing

Before running any analysis notebooks, two standalone Python scripts must be executed to download and prepare the raw datasets. These scripts are independent of the notebook pipeline and only need to be run once.

Script 1: download.py — GBM Tumor Dataset

Purpose

Downloads the human glioblastoma dataset (GSE84465, Darmanis et al. 2017, Smart-seq2) from NCBI GEO and converts it into an AnnData h5ad file for use in the pipeline.

How to Run

```
python download.py
```

What It Does

- Creates a ./data/GSE84465/ directory.
- Downloads GSE84465_RAW.tar (raw count matrices per cell) and GSE84465_GBM_All_data.csv.gz (combined expression matrix) from the NCBI FTP server using urllib.
- Extracts the tar archive to ./data/GSE84465/raw/.
- Loads the CSV into a pandas DataFrame using whitespace/tab delimiter.
- Automatically detects orientation: if the matrix has more rows than columns it is transposed to cells × genes format.
- Saves the result as data/GSE84465/GSE84465_raw.h5ad.

Key Configuration

DATA_DIR	'./data' — root directory for all downloaded files
URL_TAR	NCBI FTP URL for GSE84465_RAW.tar
URL_CSV	NCBI FTP URL for GSE84465_GBM_All_data.csv.gz
Output	data/GSE84465/GSE84465_raw.h5ad

Notes

The download skips files that already exist on disk (safe to re-run). The CSV file is large and loading it into memory can take several minutes. The safe_read_annotation() helper function attempts multiple delimiters (comma, tab, semicolon) if the default whitespace reader fails.

Script 2: cache_normal_brain.py — Normal Fetal Brain Dataset

Purpose

Subsamples 100,000 cells from the HumanFetalBrainPool HDF5 dataset and saves them as a lightweight AnnData file. This avoids loading the full multi-hundred-thousand-cell dataset into memory during integration.

Prerequisites

The full HumanFetalBrainPool.h5 file must already be present at data/HumanFetalBrainPool/HumanFetalBrainPool.h5. This file is not downloaded automatically and must be obtained separately (e.g., from the Linnarsson lab or equivalent source).

How to Run

```
python cache_normal_brain.py
```

What It Does

- Opens the HDF5 file and reads the total cell count from shoji/Expression.
- Randomly samples 100,000 cell indices without replacement (random seed 42 for reproducibility).
- Loads the expression matrix, cell IDs, gene names, and metadata (Age, Region, CellClass, Donor, TotalUMIs, NGenes, MitoFraction) for the sampled cells only — avoids loading the full dataset.
- Constructs a sparse AnnData object with obs['batch'] = 'HumanFetalBrain' and obs['condition'] = 'normal'.
- Saves the result to data/HumanFetalBrainPool/HumanFetalBrainPool_subsampled_100k.h5ad.

Key Configuration

n_cells_to_sample	100,000 — number of cells to subsample
random seed	42 — ensures reproducible subsampling
Input	data/HumanFetalBrainPool/HumanFetalBrainPool.h5
Output	data/HumanFetalBrainPool/HumanFetalBrainPool_subsampled_100k.h5ad

Notes

The subsampled file is what the scVI notebook reads as NORMAL_DATA_PATH. If you want a different sample size, change n_cells_to_sample and re-run; be aware that larger samples will increase memory usage in the integration step.

1. scVI Training (scvi_training.ipynb)

Purpose

Integrates tumor and normal brain cells using a variational autoencoder (scVI) to correct batch effects while preserving biological variation. Produces a shared latent embedding for all downstream analyses.

Key Configuration

LOAD_EXISTING_MODEL	True / False — skip training and load saved model
MODEL_PATH	'models/scvi_model_improved'
ADATA_PATH	'models/tumor_normal_integrated.h5ad'
TUMOR_DATA_PATH	Path to Darmanis GBM h5ad file
NORMAL_DATA_PATH	Path to normal fetal brain h5ad file

How to Run

- Set LOAD_EXISTING_MODEL = False to train from scratch, or True to reload a saved model.
- Run all cells sequentially. Training (~300 epochs) requires a GPU for reasonable speed.
- Outputs are saved to models/ and outputs/scvi/.

Workflow Steps

Steps 1–4: Data Loading and Preprocessing

Both datasets are loaded as AnnData objects. Quality control filters cells with fewer than 200 detected genes and flags mitochondrial genes. The common gene set between tumor and normal is identified, and 20,000 highly variable genes are selected using seurat_v3 across the combined dataset to reduce memory requirements.

Step 5: Concatenation

Datasets are merged using anndata.concat() with explicit batch labels. Raw counts are preserved in adata.layers['counts'] — this is critical because scVI requires raw, un-normalized counts.

Step 6: scVI Setup

```
scvi.model.SCVI.setup_anndata(adata_combined, layer='counts', batch_key='batch')
model = scvi.model.SCVI(adata_combined, n_layers=2, n_latent=30,
    gene_likelihood='nb', encode_covariates=True, deeply_inject_covariates=True)
```

The model uses a negative binomial likelihood, which is appropriate for count data. Covariate injection ensures batch information is used during encoding and decoding.

Step 7: Training

Training runs for up to 300 epochs with early stopping. A GPU is automatically detected and used if available. Batch size is 256.

Steps 8–9: Embeddings and Saving

After training, the 30-dimensional latent representation is extracted via model.get_latent_representation() and stored in adata.obsm['X_scVI']. Leiden clustering (resolution 0.5) is applied on the scVI neighbor graph. Both the model and the AnnData file are saved for downstream use.

Steps 10–12: Visualization and Validation

UMAP plots colored by condition, batch, and clusters check that batch effects are removed. Stem marker genes (SOX2, NES, VIM, FABP7, ASCL1, OLIG2) are plotted to verify biological signals are preserved. Cluster composition analysis shows which clusters are mixed between tumor and normal.

Outputs

models/scvi_model_improved/	Trained scVI model directory
models/tumor_normal_integrated.h5ad	Integrated AnnData with X_scVI embedding
outputs/scvi/	UMAP and marker gene plots

2. DPT Analysis (dpt_analysis.ipynb)

Purpose

Computes Diffusion Pseudotime (DPT) and PAGA trajectory analysis on tumor cells only to reconstruct the developmental hierarchy from stem-like tumor cells to differentiated states.

Key Configuration

MANUAL_ROOT_CLUSTER	Cluster ID used as pseudotime root (default: 33)
ROOT_CLUSTER	Auto or manually set root cluster
STEM_MARKERS	SOX2, NES, VIM, FABP7, ASCL1, OLIG2

How to Run

- This notebook reuses the scVI model loading code from the first notebook. Ensure LOAD_EXISTING_MODEL = True and that the saved files exist.
- Sections 1–12 are identical to the scVI notebook (data loading boilerplate). Analysis begins at Step 13.
- Adjust MANUAL_ROOT_CLUSTER to the cluster with the highest stem marker expression.

Workflow Steps

Step 13: Tumor Cell Extraction

Tumor cells are extracted from the integrated AnnData using the 'condition' label. Neighbors are recomputed specifically for the tumor-only subset using the X_scVI embedding (n_neighbors=15), which is important because the neighbor graph from the full dataset is not appropriate for trajectory analysis.

Step 14: Root Cluster Selection

The root cell cluster (representing the most stem-like state) is identified either manually via MANUAL_ROOT_CLUSTER or automatically by selecting the cluster with the highest combined expression of stem marker genes. The default cluster 33 should be verified by checking UMAP and marker expression plots.

Step 15: PAGA Analysis

PAGA (Partition-based Graph Abstraction) computes the connectivity between Leiden clusters. This reveals the coarse-grained differentiation topology. The notebook produces two views: the PAGA graph colored by cluster identity, and a PAGA graph colored by stem score to show the direction of decreasing stemness.

The PAGA-initialized layout provides a better representation of the developmental trajectory than standard UMAP:

```
sc.tl.paga(tumor_cells, groups='leiden_scvi')
sc.tl.draw_graph(tumor_cells, init_pos='paga')
```

Step 15b: Diffusion Map

A 15-component diffusion map is computed on tumor cells. Diffusion map coordinates form the basis for the DPT calculation and are visualized to verify that the leading components separate stem-like from differentiated cells.

Step 16: DPT Computation

The root cell is selected as the cell in the root cluster whose diffusion map coordinates are closest to the cluster centroid. DPT is then computed using sc.tl.dpt() with 10 diffusion components:

```
sc.tl.dpt(tumor_cells, n_dcs=10)
```

Steps 17–19: Visualization

Pseudotime is visualized on UMAP, PAGA graph, and PAGA-initialized layout. A per-cluster pseudotime distribution table shows the temporal ordering of clusters from stem-like (low pseudotime) to differentiated (high pseudotime).

Outputs

outputs/diffusion/	Stem marker expression per cluster, diffusion map plots
outputs/paga/	PAGA graph, PAGA layout, UMAP vs PAGA comparison
tumor_cells (in memory)	AnnData with dpt_pseudotime, stem_score, X_diffmap

3. Enrichment Analysis (enrichment_analysis.ipynb)

Purpose

Performs marker gene identification and pathway/transcription factor activity analysis per cluster using decoupler, PROGENy, DoRothEA, and Hallmark gene sets to characterize biological states of each tumor cell cluster.

Dependencies

```
pip install decoupler omnipath
```

How to Run

- Run data loading and tumor cell extraction cells (same as notebooks 1 and 2).
- Execute the enrichment cells in order: marker genes, PROGENy, DoRothEA, Hallmark ORA.
- Results accumulate in tumor_cells.obsm — each method writes to a different key.

Workflow Steps

Wilcoxon Marker Genes

Differentially expressed marker genes are identified per cluster using the Wilcoxon rank-sum test:

```
sc.tl.rank_genes_groups(tumor_cells, groupby='leiden_scvi', method='wilcoxon')
```

Results are visualized as a dotplot of the top 10 marker genes per cluster and saved. Per-group DataFrames can be extracted with sc.get.rank_genes_groups_df().

PROGENy Pathway Activity

PROGENy estimates the activity of 14 cancer-relevant signaling pathways (EGFR, MAPK, PI3K, WNT, p53, TNFa, etc.) using a footprint approach. The top 300 pathway target genes are used:

```
progeny = dc.op.progeny(organism='human', top=300)
dc.mt.mlm(data=tumor_cells, net=progeny, verbose=True, raw=False)
```

Results in tumor_cells.obsm['score_mlm'] are copied to obsm['progeny'] and averaged per cluster for a heatmap. Note: save results immediately after running MLM because the next method overwrites obsm['score_mlm'].

DoRothEA Transcription Factor Activity

DoRothEA estimates activity of transcription factors using confidence levels A, B, and C (highest confidence):

```
dorothea = dc.op.dorothea(organism='human', levels=['A', 'B', 'C'])
```

```
dc.mt.mlm(data=tumor_cells, net=dorothea, verbose=True, raw=False)
```

Results are saved to obsm['dorothea'] and displayed as a per-cluster heatmap showing the most active TFs.

Hallmark Gene Set ORA

Over-representation analysis is run against MSigDB Hallmark gene sets to identify enriched biological processes per cluster:

```
hallmark = dc.op.hallmark(organism='human')
dc.mt.ora(data=tumor_cells, net=hallmark, verbose=True, raw=False)
```

Important Notes on decoupler API

The decoupler library changed its API between versions. In recent versions, methods are accessed via dc.mt (methods), dc.op (omnipath), and dc.tl (tools) submodules. If using an older version, methods may be at the top level (e.g., dc.run_mlm). The notebook uses the new submodule-based API.

Outputs

outputs/enrichment/marker_genes_per_cluster.png	Top 10 marker genes per cluster
tumor_cells.obsm['progeny']	PROGENy pathway activity scores
tumor_cells.obsm['dorothea']	DoRothEA TF activity scores
tumor_cells.obsm['hallmark']	Hallmark ORA enrichment scores

4. VAE Step 1 — Baseline VAE (step1_vae_without_balanced_sampling.ipynb)

Purpose

Implements a custom Variational Autoencoder (VAE) using scvi-tools components to learn a latent embedding of all 103,426 cells. This step establishes a baseline representation before introducing contrastive learning guidance.

Key Configuration

TRAIN_MODE	True: train from scratch | False: load pre-trained model
N_LATENT	15 — latent space dimensionality
N_HIDDEN	256 — hidden layer size
N_LAYERS	2 — encoder/decoder depth
N_EPOCHS	150 — maximum training epochs
BATCH_SIZE	256
LEARNING_RATE	1e-3
PATIENCE	20 — early stopping patience
MODEL_PATH	'models/best_vae_model.pt'
ADATA_PATH	'models/tumor_normal_integrated.h5ad'

How to Run

- Set TRAIN_MODE = True for first run. Set to False to reload a saved checkpoint.
- The notebook has two sections: a class/function definition section (Parts 1–5) and a pipeline execution section. Run Parts 1–5 first to define classes, then the pipeline.

Model Architecture

scVAE_scvi Class

The VAE uses scvi-tools' Encoder and DecoderSCVI components, which include batch normalization and dropout. The generative model uses either a Zero-Inflated Negative Binomial (ZINB) or Normal distribution:

```
class scVAE_scvi(nn.Module):
    # Encoder: genes → (mu, log_var) in latent space
```

```
    self.encoder = Encoder(n_genes, n_latent, n_layers, n_hidden, ...)
    # Decoder: latent → ZINB parameters (px_rate, px_r, px_dropout)
    self.decoder = DecoderSCVI(n_latent, n_genes, n_layers, n_hidden, ...)
```

The loss combines reconstruction (ZINB log-likelihood or MSE) with KL divergence to the standard normal prior. Library size is estimated from the input counts to scale the reconstruction.

Training Function

The train_vae() function includes gradient clipping (clip=1.0) and early stopping. Loss history is tracked separately for train and test sets. The KL weight follows a warm-up schedule to stabilize early training.

Workflow Steps

Steps 1–2: Data Preparation and Model Initialization

Raw counts from adata.layers['counts'] are converted to PyTorch tensors and split into train (90%) and test (10%) DataLoaders. The model is initialized with the parameters above.

Steps 3–4: Training and Visualization

The model trains for up to 150 epochs with early stopping. Training and test loss curves are plotted via plot_training_history() to verify convergence and check for overfitting.

Steps 5–6: Latent Space Extraction and UMAP

The full dataset is encoded using the trained encoder's mean (mu) output. The 15-dimensional latent coordinates are stored in adata.obsm['X_vae'] and used to compute a UMAP for visualization.

Stem Marker Visualization

SOX2, NES, VIM, OLIG2, FABP7, ASCL1, PROM1, CD44, NOTCH1, HES1, PAX6, and BMI1 are overlaid on the VAE-based UMAP to assess how well the embedding separates stem-like cells.

Outputs

models/best_vae_model.pt	Trained VAE model weights
data/adata_with_vae_latent.h5ad	AnnData with X_vae embedding
outputs/vae_figures_step1/	Training curves and UMAP plots

5. VAE Step 2 — Stem Marker Similarity (step2_stem_markers_only.ipynb)

Purpose

Computes cell-cell cosine similarity based exclusively on stem/progenitor marker gene expression. The resulting sparse similarity matrix is used in Step 3 to guide contrastive learning toward biologically meaningful cell relationships.

Why Stem Markers Only?

Using all 16,388 genes would dilute the stemness signal with housekeeping and irrelevant genes. By restricting to 16 curated stem markers, the similarity matrix directly captures the stemness gradient from tumor stem cells to differentiated normal cells, which is the key axis for trajectory analysis.

Stem Marker Genes Used

Core NSC	SOX2, NES, VIM, FABP7, ASCL1, OLIG2
Additional progenitor	PROM1 (CD133), CD44, NOTCH1, HES1, PAX6, BMI1
GBM-specific	EGFR, PDGFRA, CHI3L1, TERT

How to Run

- Requires models/tumor_normal_integrated.h5ad.
- All cells (103K) are processed — expect runtime of 10–30 minutes depending on hardware.
- Key parameter: top_k=1000 and chunk_size=5000 in calculate_topk_similarity().

Workflow Steps

Part 1–2: Marker Definition and Availability Check

Stem marker genes are defined in a dictionary. The check_marker_availability() function performs case-insensitive matching against the dataset's gene names and reports which markers are present.

Part 3: Expression Extraction

The extract_marker_expression() function retrieves raw counts for available markers, normalizes to CPM (counts per million), and log-transforms for numerical stability. The output is a dense matrix of shape (n_cells, n_markers).

Part 4: Top-K Cosine Similarity

A full cell × cell similarity matrix would require ~40 GB RAM at 103K cells. Instead, a memory-efficient chunked approach keeps only the top-1000 most similar cells per cell (~800 MB):

```
similarity_topk = calculate_topk_similarity(
    marker_expr, top_k=1000, chunk_size=5000
)
```

The function processes cells in chunks of 5,000, computing cosine similarity within each chunk against all others. The result is stored as two arrays: indices (n_cells × 1000) and values (n_cells × 1000).

Part 5: Visualization

The plot_similarity_analysis() function shows the distribution of similarity scores, the fraction of tumor cells' top neighbors that are normal (cross-population similarity), and a similarity heatmap on a random sample of 1,000 cells.

Part 6: Save for Step 3

Results are serialized to a pickle file containing marker definitions, expression matrix, and the sparse similarity structure:

```
pickle.dump(step2_results,
open('models/cosine_similarity_results/step2_stem_similarity.pkl', 'wb'))
```

Outputs

models/cosine_similarity_results/step2_stem_similarity.pkl	Similarity indices and scores
outputs/cell_similarity_based_on_marker/	Visualization plots

6. VAE Step 3 Ultra — VAE + Contrastive (step3_vae_ultra_optimized.ipynb)

Purpose

Trains a VAE with an additional batch-aware contrastive loss term that pulls stem-similar cells together and pushes dissimilar cells apart in the latent space. This produces a biologically guided embedding (stored as X_vae_contrastive) optimized for trajectory analysis.

Key Configuration

TRAIN_MODE	True: train | False: load & visualize
ADATA_PATH	'models/tumor_normal_integrated.h5ad'
SIMILARITY_PATH	'models/cosine_similarity_results/step2_stem_similarity.pkl'
MODEL_PATH	'models/vae_contrastive_model.pt'
N_LATENT	15
N_HIDDEN	256
N_LAYERS	2
N_EPOCHS	100
BATCH_SIZE	256
LEARNING_RATE	1e-3
LAMBDA_CONTRASTIVE	Weight controlling contrastive vs reconstruction loss balance

How to Run

- Requires both the integrated AnnData (Step 1) and the similarity pickle (Step 2).
- Set TRAIN_MODE = True for initial training; False to reload and visualize.
- Monitor the loss breakdown plots to ensure reconstruction and contrastive losses both decrease.

Key Components

VAE Model

Identical scVAE_scvi architecture as Step 1 (Encoder + DecoderSCVI with ZINB likelihood).

BatchAwareContrastiveLoss

The contrastive loss operates on mini-batches and uses a fully vectorized implementation. The key insight is computing negative pairs without masking loops:

```
# For each cell, positives are its top-k similar cells from Step 2
# neg_sum = total_sum - self_similarity - positive_sum
# This avoids all masking operations and is fully GPU-accelerated
```

Similarity values from Step 2 are used as soft positive/negative weights rather than hard binary labels, which provides a more graded training signal.

Combined Training Loss

```
total_loss = reconstruction_loss + kl_weight * kl_loss + lambda_contrastive *
contrastive_loss
```

A learning rate scheduler (ReduceLROnPlateau) reduces the learning rate when validation loss plateaus. Gradient clipping is applied to prevent instability.

Outputs

models/vae_contrastive_model.pt	Trained model weights
adata.obsm['X_vae_contrastive']	15-dim contrastive-guided latent space
outputs/vae_contrastive/X_vae_contrastive_opt.npy	Saved latent coordinates
outputs/vae_contrastive/condition_plot.png	UMAP colored by condition
outputs/vae_contrastive/stem_markers.png	UMAP with marker expression overlay

7. VAE Step 3 Alternative — Pure Contrastive (step3_pure_contrastive_optimized.ipynb)

Purpose

An alternative to Step 3 Ultra that removes the VAE decoder entirely, using only a feedforward encoder trained with contrastive loss. This is simpler and faster than the VAE variant and can serve as a comparison baseline.

Difference from Step 3 Ultra

Step 3 Ultra	VAE (encoder + ZINB decoder) + contrastive loss
Step 3 Pure Contrastive	Encoder only — no decoder, no reconstruction loss
Loss function	Contrastive loss only (no KL, no ZINB)
Output key	adata.obsm['X_pure_contrastive']

How to Run

- Same prerequisites as Step 3 Ultra (integrated AnnData and Step 2 similarity pickle).
- Faster training since no ZINB decoder is computed.
- Compare UMAP visualizations of X_vae_contrastive and X_pure_contrastive to select the better embedding.

SimpleEncoder Architecture

A stack of linear layers with batch normalization and dropout, without any probabilistic components:

```
class SimpleEncoder(nn.Module):
    # Input: raw counts (n_genes)
    # Output: latent coordinates (n_latent)
    # No mu/log_var split — deterministic encoding
```

Training

Only BatchAwareContrastiveLoss is used, with the same vectorized implementation as Step 3 Ultra. A ReduceLROnPlateau scheduler halves the learning rate after 10 epochs without improvement.

Outputs

adata.obsm['X_pure_contrastive']	Contrastive encoder latent space
outputs/pure_contrastive/X_pure_contrastive.npy	Saved latent coordinates
outputs/pure_contrastive/pure_contrastive_condition.png	UMAP by condition
outputs/pure_contrastive/stem_markers.png	Stem marker UMAP overlay

8. scDiffEq — Neural SDE Trajectories (scdiffeq_tumor_analysis.ipynb)

Purpose

Applies scDiffEq (stochastic differential equation model) to simulate cell trajectories from stem-like progenitors through differentiation. Models drift (deterministic direction of differentiation) and diffusion (stochastic noise) as functions of the cellular state.

Dependencies

```
pip install scdiffeq pytorch-lightning
```

Key Configuration

data_path	'models/tumor_cells_with_dpt.h5ad' — tumor AnnData with DPT
time_key	'time_bin' — discretized pseudotime for scDiffEq
n_latent_dims	30
BATCH_SIZE	512
LEARNING_RATE	1e-5
N_EPOCHS	250 (fit call uses 1500 epochs)
ckpt_path	Path to a saved checkpoint for model reloading

How to Run

- Requires tumor_cells_with_dpt.h5ad — the tumor AnnData with dpt_pseudotime and stem_score columns from the DPT notebook.

- PCA is computed if not already present (required by scDiffEq as the input representation).
- scDiffEq is optional — the notebook checks for import availability and prints a warning if unavailable.
- Training saves checkpoints to LightningSDE-FixedPotential-RegularizedVelocityRatio/. Reload with sdq.io.load_diffeq().

Workflow Steps

Section 1–3: Data Loading and Quality Check

The tumor AnnData is loaded and validated. Required columns (dpt_pseudotime, stem_score) and representations (X_pca, X_scVI) are verified. PCA is computed if missing.

Section 4–5: Biological Time Binning

DPT pseudotime is discretized into biological time bins that respect cell density. Bins preserve the accumulation pattern of cells at each developmental stage, which is important for the SDE model to correctly weight transitions:

```
# Continuous pseudotime → discrete time_bin variable
# Used as the time axis for scDiffEq training
```

Section 6: Model Initialization and Training

```
model = sdq.scDiffEq(
    adata=adata_tumor, time_key='time_bin',
    batch_size=512, train_lr=1e-5
)
model.fit(train_epochs=1500, train_lr=1e-5, monitor='epoch_validation_loss')
```

Section 7–8: Drift and Diffusion Extraction

model.drift() and model.diffusion() compute per-cell drift and diffusion magnitudes. These are added to adata.obs and visualized on UMAP to show which cells are undergoing rapid state transitions (high drift) vs. stable states (low drift, low diffusion).

Sections 9–10: Progenitor Selection and Trajectory Simulation

Stem-like progenitor cells are selected as those in the top 10% of stem score and bottom 25% of pseudotime. Trajectories are simulated from these starting points:

```
adata_sim = sdq.tl.simulate(
    adata_tumor, idx=sampled_progenitors,
    N=200, diffeq=model.DiffEq, dt=0.1, device='cuda'
)
```

Sections 11–13: Trajectory Annotation and Visualization

Simulated cells are annotated with cluster labels from nearest neighbors in PCA space. Trajectories are projected onto the UMAP embedding and plotted per starting progenitor.

Temporal dynamics (drift and diffusion along simulation time) are computed for the simulated cells.

Outputs

LightningSDE-.../checkpoints/	PyTorch Lightning training checkpoints
adata_tumor.obs['drift']	Per-cell drift magnitude
adata_tumor.obs['diffusion']	Per-cell diffusion magnitude
adata_sim	Simulated trajectory AnnData

9. CONCORD Embeddings (concord_tumor_normal_embeddings.ipynb)

Purpose

Uses CONCORD (Contrastive Learning for Single-Cell Data Integration) to produce a batch-corrected embedding of the full tumor + normal dataset. CONCORD operates on log-normalized data and uses domain labels for contrastive batch integration, providing an alternative to the scVI-based embedding.

Dependencies

```
pip install concord plotly
```

Key Configuration

DATA_PATH	'models/tumor_normal_integrated.h5ad'
n_top_features	3000 — HVGs for CONCORD input (increase to 4000–5000 for complex data)
domain_key	'condition' — batch label for CONCORD integration
device	Auto-detected (CUDA or CPU)
preload_dense	True — set to False if memory errors occur at 103K cells
OUTPUT_DIR	'outputs/concord_images'

How to Run

- Set adata.X to log1p_norm before running CONCORD (the notebook does this automatically, keeping raw counts safe in adata.layers['counts']).
- Run cells sequentially from Section 1 through 10.
- The CONCORD fit_transform step is the most time-consuming (~5–20 min depending on hardware).

Important: domain_key Selection

'dataset'	Recommended — most granular; separates tumor vs normal data sources
'batch'	Use if single experiment with technical batches
'condition'	Never use — this is biological signal to be preserved, not corrected

Workflow Steps

Section 3: Set adata.X to log1p_norm

CONCORD reads from adata.X. The raw counts must remain in adata.layers['counts']. The notebook backs up the current X, then sets X to the log-normalized layer:

```
adata.layers['X_backup'] = adata.X.copy()
adata.X = adata.layers['log1p_norm'].copy()
```

Section 4: Feature Selection

CONCORD's built-in HVG selection is used to select 3,000 features via seurat_v3 variance ranking. An optional batch-aware approach using scanpy is commented out for reference.

Section 6: Running CONCORD

```
cur_ccd = ccd.Concord(
    adata=adata, input_feature=feature_list,
    domain_key='condition', device=device, preload_dense=True
)
cur_ccd.fit_transform(output_key='Concord')
```

The embedding is stored in adata.obsm['Concord'] (default dimensionality ~64).

Section 7: 2D UMAP

A UMAP is computed on the CONCORD embedding using cosine metric with n_neighbors=30 and min_dist=0.1. Results are stored in adata.obsm['Concord_UMAP'].

Section 8–9: Visualization

UMAP plots are generated colored by condition, leiden_scvi, batch, dataset, and stem marker genes (SOX2, NES, VIM, FABP7, ASCL1, OLIG2). Integration quality is assessed by checking that batches mix in the embedding while biological conditions remain separated.

Section 10: Leiden Clustering on CONCORD Embedding

Leiden clustering is re-run on the CONCORD neighbor graph and compared against the scVI-based clustering with a cross-tabulation to assess consistency between the two integration methods.

Outputs

adata.obsm['Concord']	CONCORD embedding (~64 dimensions)
adata.obsm['Concord_UMAP']	2D UMAP of CONCORD embedding
adata.obs['leiden_concord']	Leiden clusters from CONCORD graph
outputs/concord_images/	All UMAP and marker visualization PNGs

Appendix: File and Data Flow Summary

Shared Files

models/tumor_normal_integrated.h5ad	Central AnnData — written by scVI, read by all others
models/scvi_model_improved/	scVI model — written by scVI, read by DPT and Enrichment
models/tumor_cells_with_dpt.h5ad	Tumor subset with DPT — read by scDiffEq
models/cosine_similarity_results/step2_stem_similarity.pkl	Similarity matrix — written by Step 2, read by Step 3

AnnData Key Reference

adata.X	Log-normalized expression (or raw counts depending on step)
adata.layers['counts']	Raw UMI counts — preserved throughout pipeline

adata.layers['log1p_norm']	Log1p-normalized counts
adata.obsm['X_scVI']	30-dim scVI batch-corrected embedding
adata.obsm['X_vae']	15-dim VAE Step 1 baseline embedding
adata.obsm['X_vae_contrastive']	15-dim VAE + contrastive embedding
adata.obsm['X_pure_contrastive']	15-dim pure contrastive encoder embedding
adata.obsm['Concord']	CONCORD integration embedding
adata.obs['condition']	'tumor' or 'normal'
adata.obs['leiden_scvi']	Leiden clusters from scVI embedding
adata.obs['dpt_pseudotime']	Diffusion pseudotime (tumor cells only)
adata.obs['stem_score']	Mean expression of stem marker genes

GPU Notes

scVI training, scDiffEq training and simulation, and CONCORD all benefit from CUDA GPUs. The scVI and CONCORD notebooks auto-detect GPU availability. scDiffEq simulation explicitly uses device='cuda' in the sdq.tl.simulate() call — change to 'cpu' if no GPU is available.

Memory Notes

The full dataset of ~103K cells requires approximately 16–32 GB RAM for in-memory processing. The Step 2 similarity computation uses chunked processing (chunk_size=5000) to avoid memory overflow. The scDiffEq simulation stores trajectory results in a separate AnnData object to avoid inflating the main object.